

Structured Data Integration (2)

TECHNOLOGIES FOR INFORMATION SYSTEMS

Letizia Tanca

Politecnico di Milano



The steps of Data Integration: how is it done?



Schema Reconciliation

Schema reconciliation: mapping the **data structure**

Record Linkage

Record linkage: data matching based on **the same content**

Data Fusion

Data fusion: reconciliation of **non-identical content**

Be there a schema or not, we may have inconsistencies in the data

At query processing time, when a real-world object is represented by instances in different databases, they may have different values.

- Record Linkage (aka Entity Resolution): finding the info that refer to same real-world entities.

- Data Fusion: once recognized that two items refer to the same entity, how do we reconcile inconsistent information?

Record Linkage (aka Entity Resolution)

We have understood already that **data integration often needs finding strings that refer to the same real-world entities:**

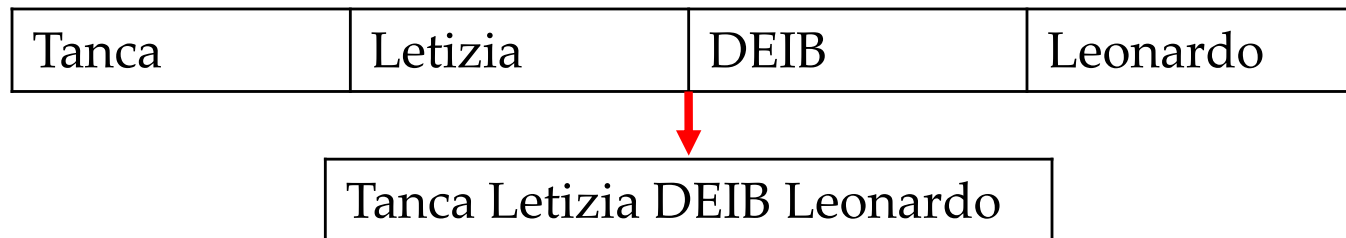
- “Politecnico di Milano” and “Politecnico Milano”
- “Via ponzio 34” and “Via Ponzio 34/5”

Whatever the data model, **we have to recognize when two datasets contain the same information**

Note: Again we refer again to the relational data model, because it is the most common case and also because **the problem is pretty much the same**

String matching vs. tuple or (more generally) structured-data matching

Consider the case of relational databases. One possibility is to compare two tuples by transforming them into strings of characters:



But if I compare that with:

Tanca Letizia DEIS Lonardo

it will be difficult to see that they refer to the same person!

Therefore, If the fields are distinct, it is easier to spot similarities, e.g., that a word like “Lonardo” is a wrong representation of “Leonardo”, because we know that Leonardo is a campus of Politecnico. The same would happen with “DEIS” and “DEIB”.

We wouldn't be able to teach this knowledge to the system if the tuple were treated as a single string

Let us start by matching strings

- Typing errors, e.g., Giovanni Rossi vs. Givanni Rossi
- Different representation conventions, e.g., Sept 20th vs. 20/9
- Nicknames or abbreviations, e.g., Giuseppe vs. Beppe
- String inversion, e.g., Giovanni Rossi vs Rossi, Giovanni
- Abbreviations, e.g. Politecnico di Milano vs PoliMI

String similarity and similarity measures

- Given two sets of strings, find all pairs from the two sets that refer to the same real-world entity.
- Each of these pairs is called a **match**
- In order to find matches, we can use a **similarity measure**: a value $s(x,y)$ in $[0,1]$: $s(x,y)=1$ iff $x=y$, and the closer $s(x,y)$ is to 0, the lower is the similarity.
- We can decide that x and y match if $s(x,y) \geq t$, with $0 \leq t \leq 1$

Types of similarity measures:

- ✓ **Sequence-based**: edit distance, Needleman-Wunch, affine gap, Smith-Waterman, Jaro, Jaro-Winkler
- ✓ **Set-based**: overlap, Jaccard, TF/IDF
- ✓ **Hybrid**: generalized Jaccard, soft TF/IDF, Monge-Elkan
- ✓ **Phonetic**: Soundex

We will learn some of them

- **Note**: using Distance Measures does not change the results: the lower the similarity, the higher the distance....

Edit (or Levenshtein) Distance

- The **edit distance** is based on the minimal number of operations that are needed to transform string *a* into string *b*: *character insertion, character deletion, character replacement*
- Example:

a = Politecnico di Milano, *b* = Politecnico Milano

$d(a,b) = 2$, using the following sequence of operations:

1.Delete the character **d** of string *a*

2.Delete the character **i** of string *a*

Similarity: $s(a,b) = 1 - d(a,b) / [\max(\text{length}(a), \text{length}(b))]$

Example: $s(\text{Politecnico di Milano}, \text{Politecnico Milano}) =$

$$1 - 2 / \max(19, 17) = 1 - 0,105... = 0,894... \rightarrow \text{acceptable!}$$

Set-based Similarity Measures

View strings as *multisets of tokens*: this measure corresponds to dividing the strings into *tokens*, and computing the measure on the two sets of tokens. The tokens are pieces of the strings, like for instance, we can choose tokens of length 2. E.g., for strings *pino* and *pin* :

$$A = \{\#p, pi, in, no, o\# \}, \quad B = \{\#p, pi, in, n\# \}$$

An example of a set-based measure is the *Jaccard measure*:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

$$J(\text{pino}, \text{pin}) = 3/(5+4)-3 = 3/6 = 1/2$$

Phonetic Similarity Measures

- Match strings based on their *sound*
- **Soundex** is the most common one. Soundex calculates a *four-character code* from a word based on the pronunciation and considers two words as similar if their codes are equal.
- Similar sounding letters are assigned the same *soundex code*.
- Used in archives, e.g., searching ancestors, since often the way the names of the families evolve is according to sound, like for instance **Legoff** and **Legough**.
- What is an advantage for the previous use can be a disadvantage for common words, like e.g., “coughing” and “coffin”, that are pronounced almost the same in English.
- Strongly based on the language, since the way words are pronounced is fundamental, e.g., the string “gn” in English and Italian is pronounced in different ways.

Efficiency

- Applying a similarity measure $s(x,y)$ to all pairs is quadratic in the size of the datasets
- Many solutions have been proposed to choose the pairs of words that most probably match.

Now record matching

Types of record matching

- Rule-based
- Learning- based (supervised or unsupervised)
- Probabilistic

Blocking: a technique applied before applying the proper matching rules.

Modern Entity Linkage methods, in order to reduce the dimension of the problem, partition the initial search space into considerably small *blocks* of records that can be considered similar according to some metrics, comparing then only the records belonging to corresponding blocks, and thus greatly reducing the complexity of the algorithm (sum of the squares of all the block sizes)

Rule-based Matching

SSN	NAME	AGE	SALARY	POSITION
123456789	JOHN	34	30K	ENGINEER
234567891	KETTY	27	25K	ENGINEER
345678912	WANG	39	32K	MANAGER

Many types of rules exist.

E.g.

$\text{sim}(x,y) =$

$$0.5s_{\text{SSN}}(x,y) + 0.2s_{\text{name}}(x,y) + 0.1s_{\text{age}}(x,y) + 0.1s_{\text{salary}}(x,y) + 0.1s_{\text{position}}(x,y)$$

Note: possibly choosing the most appropriate similarity measure for each attribute !

Manually written rules that specify when two tuples match. E.g. two tuples refer to the same person if they have the same SSN

SSN	NAME	AGE	SALARY	PHONE
234567891	KETTY	25	20K	1234567
345678912	WANG	38	22K	2345678
456789123	MARY	42	34K	3456789

- Rules are manually specified, it requires a lot of time and effort
- A variant is to learn rules

Learning Matching Rules

- It can be supervised or unsupervised
- **Supervised** (e.g. classification): learn how to match from training data, then apply it to match new tuple pairs:
 - Learn how to match each attribute of the tuples (the training phase): each (x_i, y_i) is a pair of elements and I_i is a label: “yes” if x_i matches y_i and “no” otherwise
 - Define the weight of each attribute in the final matching of the whole records
 - Apply the learned model to the new tuple pairs
 - Supervised learning **requires a lot of training data**
- **Unsupervised** learning (typically clustering, based on clusterizing similar values) may solve this problem

Probabilistic Matching

- Model the matching domain using a **probability distribution**
- Reason with the distribution to make matching decisions
- Key benefits
 - provide a principled framework that can naturally incorporate a variety of domain knowledge
 - can leverage the wealth of probabilistic representation and reasoning techniques already developed in the AI and DB communities
 - provide a frame of reference for comparing and explaining other matching approaches
- Disadvantages
 - computationally expensive
 - often hard to understand and debug matching decisions

Data Fusion

SSN	NAME	AGE	SALARY	POSITION
123456789	JOHN	34	30K	ENGINEER
234567891	KETTY	27	25K	ENGINEER
345678912	WANG	39	32K	MANAGER

SSN	NAME	AGE	SALARY	PHONE
234567891	KETTY	25	20K	1234567
345678912	WANG	38	22K	2345678
456789123	MARY	42	34K	3456789

Once you have understood that some data clearly **represent the same entity** (same person, in this case), you still have to cope with the problem of **what to do when some parts of the info do not match** (e.g., the SALARY)

Resolution function

Inconsistency may depend on different reasons:

- One (or both) of the sources are incorrect
- Each source has a correct but partial view, e.g. databases from different workplaces, e.g.:
 - the full salary is the sum of the two
 - the list of authors of a book is incomplete
 - one of two full names only contains the first letter of the middle name
- Often, the correct value may be obtained as a **function** of the original ones, e.g. : $value_1 + value_2$, or $1*value_1 + 0*value_2$ or $0,5*value_1 + 0,5*value_2$)

RESOLUTION FUNCTION: EXAMPLE

SSN	NAME	AGE	SALARY	POSITION
123456789	JOHN	34	30K	ENGINEER
234567891	KETTY	27	25K	ENGINEER
345678912	WANG	39	32K	MANAGER

SSN	NAME	AGE	SALARY	PHONE
234567891	KETTY	25	20K	1234567
345678912	WANG	38	22K	2345678
456789123	MARY	42	34K	3456789

SSN	NAME	AGE	SALARY	POSITION	PHONE
123456789	JOHN	34	30K	ENGINEER	NULL
234567891	KETTY	27	45K	ENGINEER	1234567
345678912	WANG	39	54K	MANAGER	2345678
456789123	MARY	42	34K	NULL	3456789

$R = \text{MAX_AGE, SUM_SALARY (R1 OuterJoin R2)}$

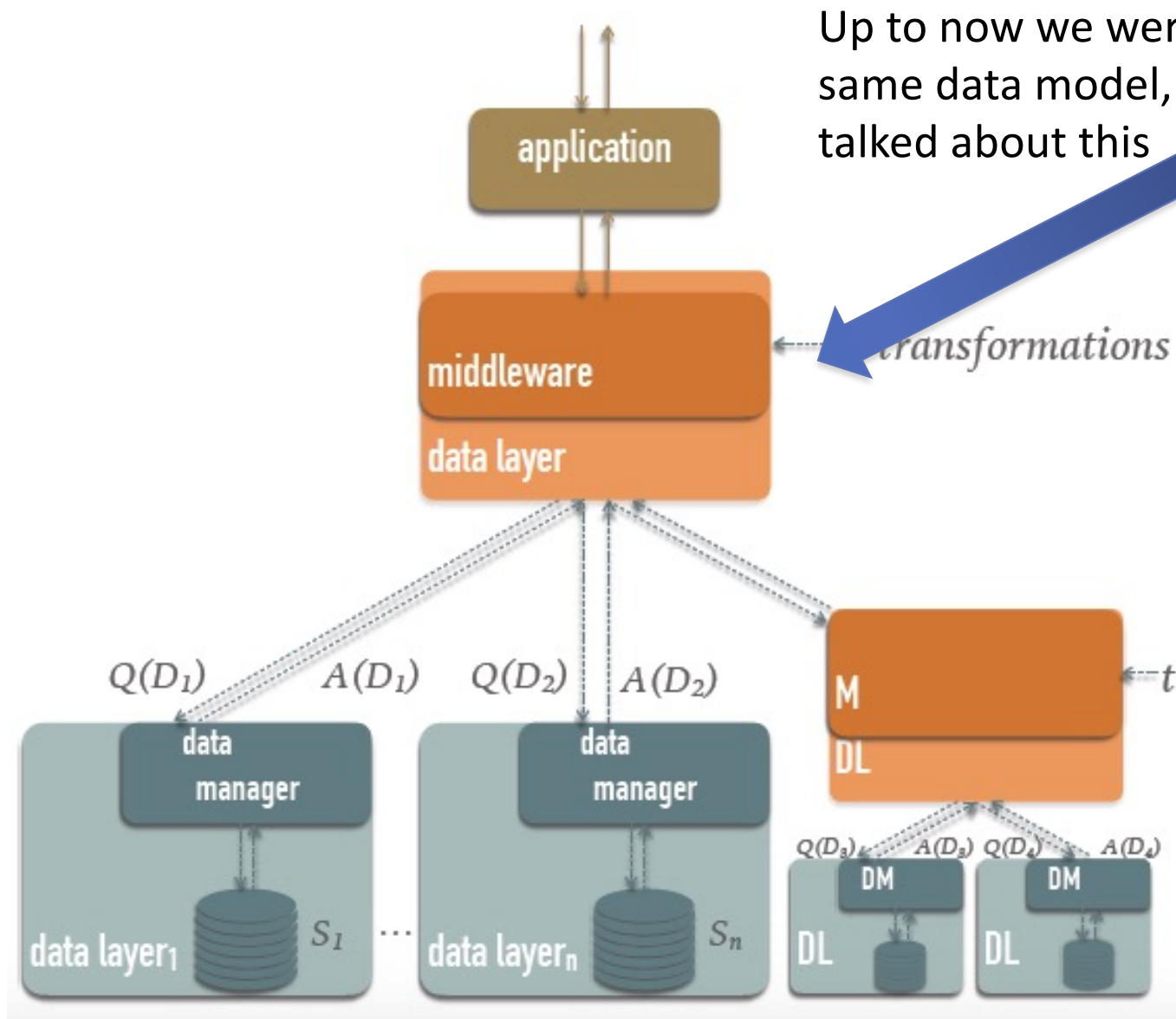
We can now start reasoning on the new application context (recall)

- A (possibly large) number of data sources
 - Heterogeneous data sources
- Different levels of data structure
 - Databases (relational, OO...)
 - Semi-structured data sources (XML, HTML, more markups ...)
 - Unstructured data (text, multimedia etc...)
- Different terminologies and different operational contexts
- Time-variant data (e.g. WEB)
- Mobile, transient data sources

Levels of source heterogeneity

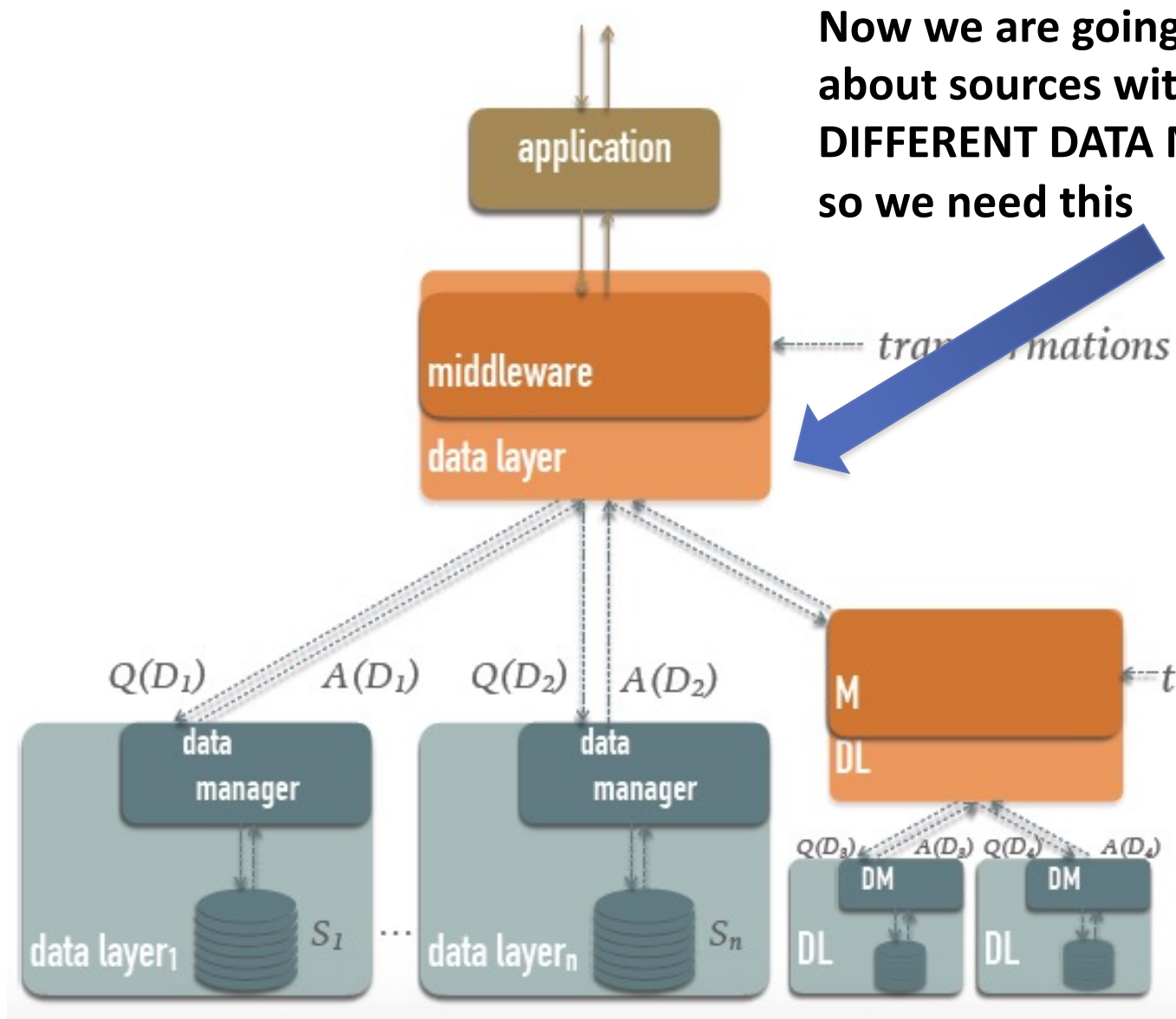
- Schemata (already seen)
- **Data Models**
- Systems (easily overcome if we reconcile data models and schemata)

Recall: A general framework for Data Integration



(Alvaro Fernandez,
EDBT Summ.
School 2017)

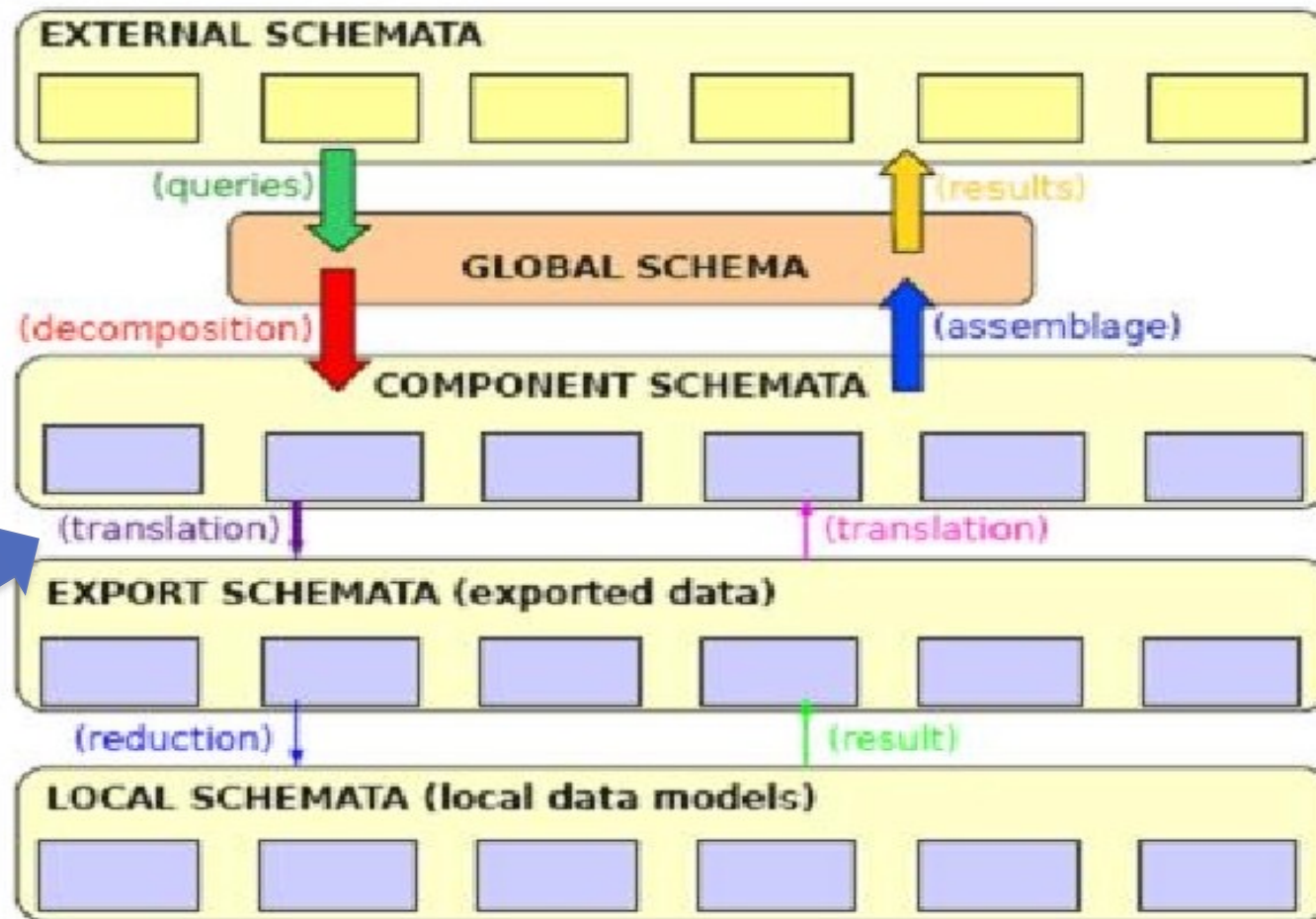
Recall: A general framework for Data Integration



(Alvaro Fernandez,
EDBT Summ.
School 2017)

Data integration in the MULTIDATABASE

With
DIFFERENT
DATA
MODELS,
we need
this



A new element in this figure:

WRAPPERS (translators)

- Wrappers basically implement the data layer of the global figure shown before: they **convert queries to the global schema into queries/commands which are understandable for the specific data source**, possibly extending the query possibilities of a data source (e.g. type conversions, we'll speak about it later on)
- They **convert query results from the source format to a format which is understandable for the users of the global schema**
- Easy to produce in the case of structured data: e.g. for data in the Relational Model and the Object Oriented Model, we can translate the queries and the results between the two models
- More difficult is the problem **if the data is not structured** (later)

Design steps

1. Reverse engineering (i.e. production of the conceptual schema)
2. Conceptual schemata integration
3. Choice of the target logical data model (e.g. relational) and translation of the global conceptual schema into that model
4. Definition of the language translation (wrapping)
5. Definition of the data views (as usual) between the translated schemas of the sources and the global schema

Bibliography

- A. Doan, A. Halevy and Z. Ives, Principles of Data Integration, Morgan Kaufmann, 2012
- L. Dong, D. Srivastava, Big Data Integration, Morgan & Claypool Publishers, 2015
- Roberto De Virgilio, Fausto Giunchiglia, Letizia Tanca (Eds.): Semantic Web Information Management – A Model-Based Perspective. Springer 2009, ISBN 978-3-642-04328-4
- M. Lenzerini, Data Integration: A Theoretical Perspective, Proceedings of ACM PODS, pp. 233-246, ACM, 2002, ISBN: 1-58113-507-6
- Clement T. Yu, Weiyi Meng, Principles of Database Query Processing for Advanced Applications , Morgan Kaufmann, 1998, ISBN: 1558604340